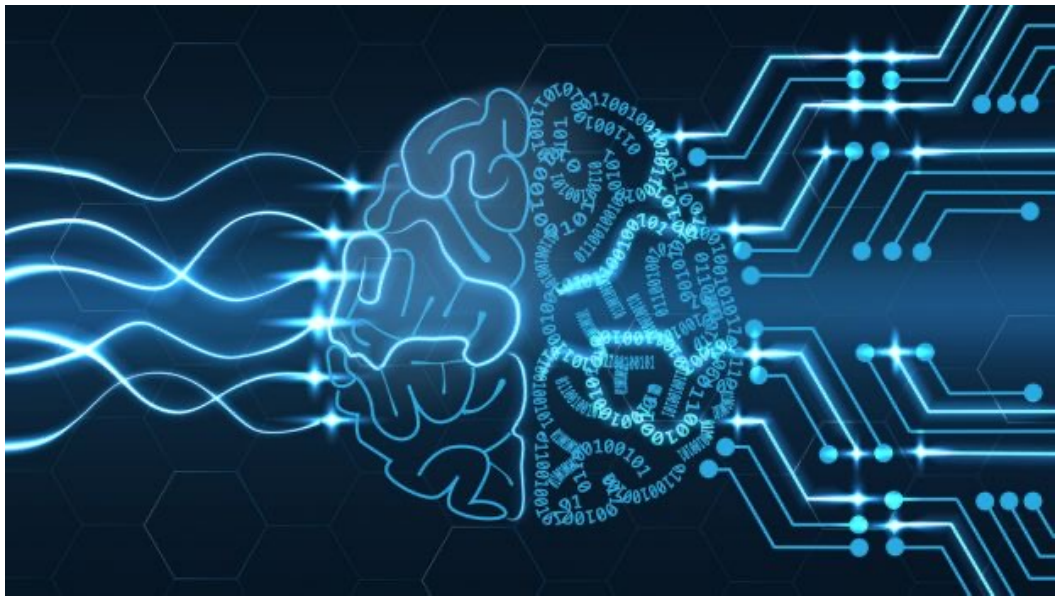


Chapter 5: Special Neural Networks

Changcheng Huang

Department of Systems and Computer Engineering

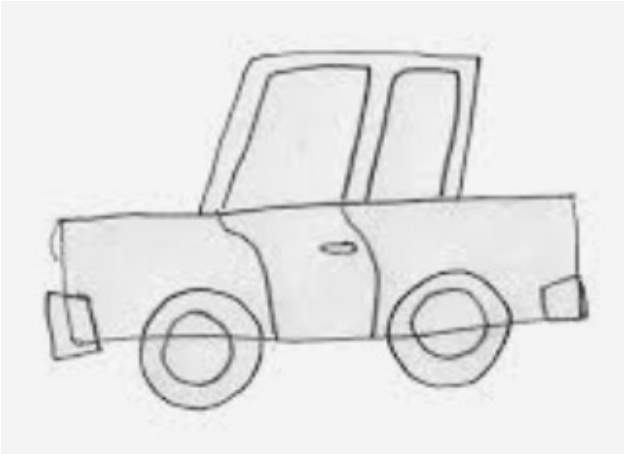
Outline

- Convolutional Neural Networks
- Recurrent Neural Networks
- Practical Methodology

Outline

- Convolutional Neural Networks
- Recurrent Neural Networks
- Practical Methodology

Image Recognition



Equivariance

Equivariance

- Equivariant representation

$$f\big(g(X)\big) = g(f(X))$$

- Example
  - shift input is the same as shift output
    - Detect edges at different locations

Convolutional Neural Network

- Convolution in one dimension

$$s(t) = (x * k)t$$

$$= \sum_{a=-\infty}^{\infty} x(a)k(t-a) = \sum_{a=-\infty}^{\infty} x(t-a)k(a)$$

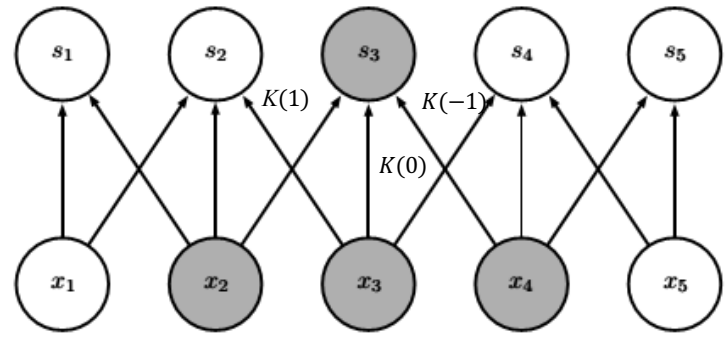
- Sparse interactions
- Parameter sharing
- Replace large matrix multiplication with convolution

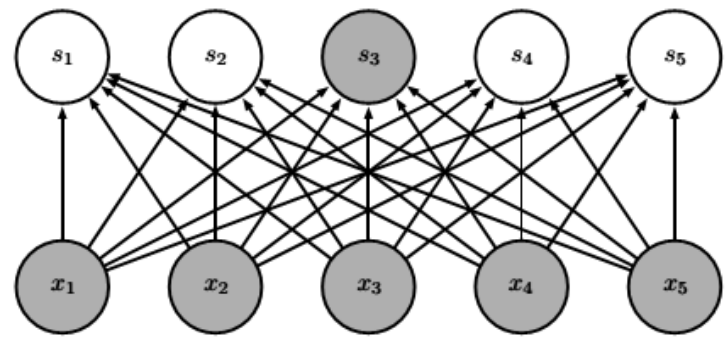
$$s(i,j) = (X * K)(i,j)$$

$$= \sum_m \sum_n x(i-m, j-n)K(m,n)$$

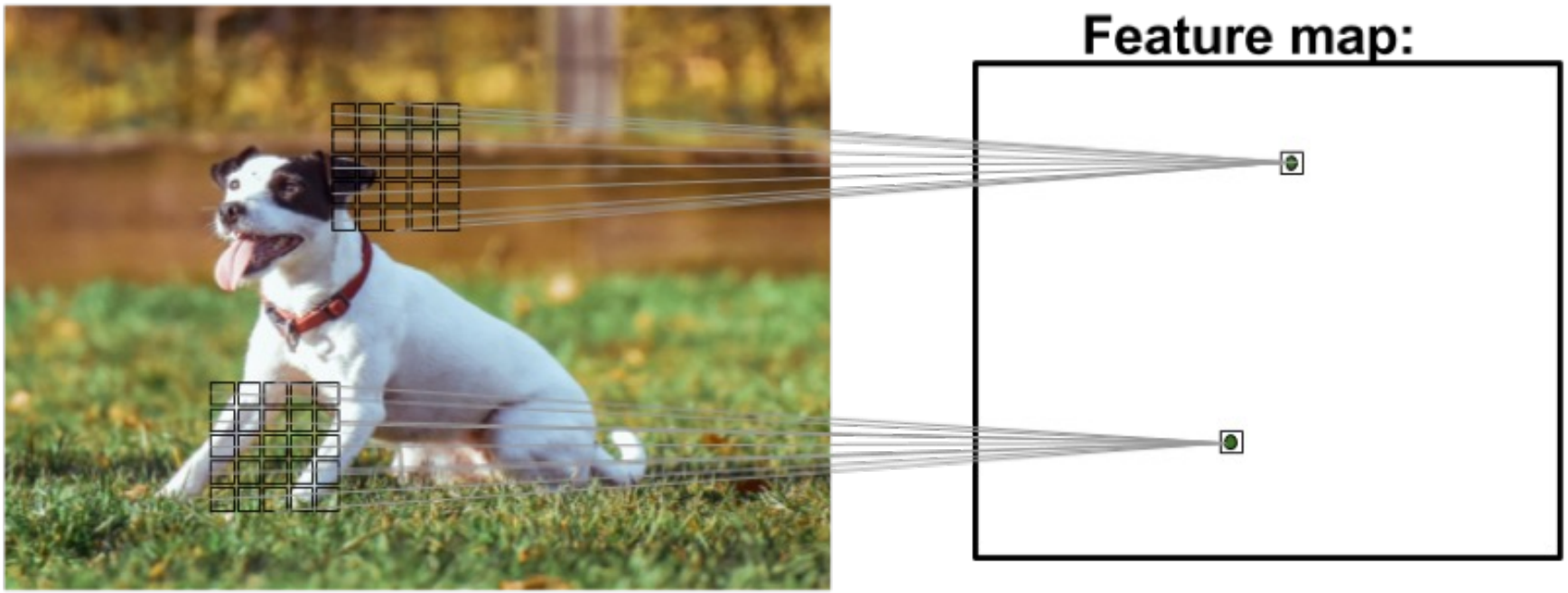
$$s(i,j) = (X * K)(i,j)$$

$$= \sum_m \sum_n x(i+m, j+n)K(m,n)$$





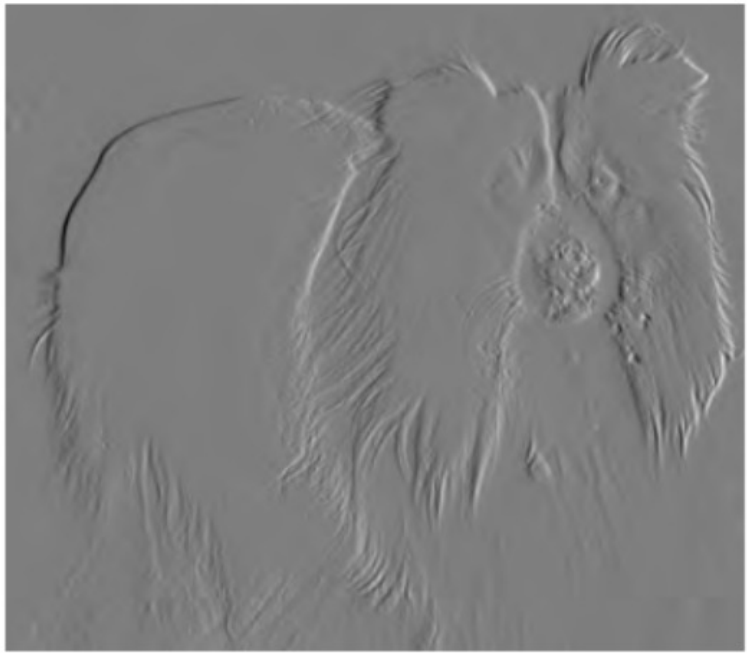
Convolutional Neural Networks



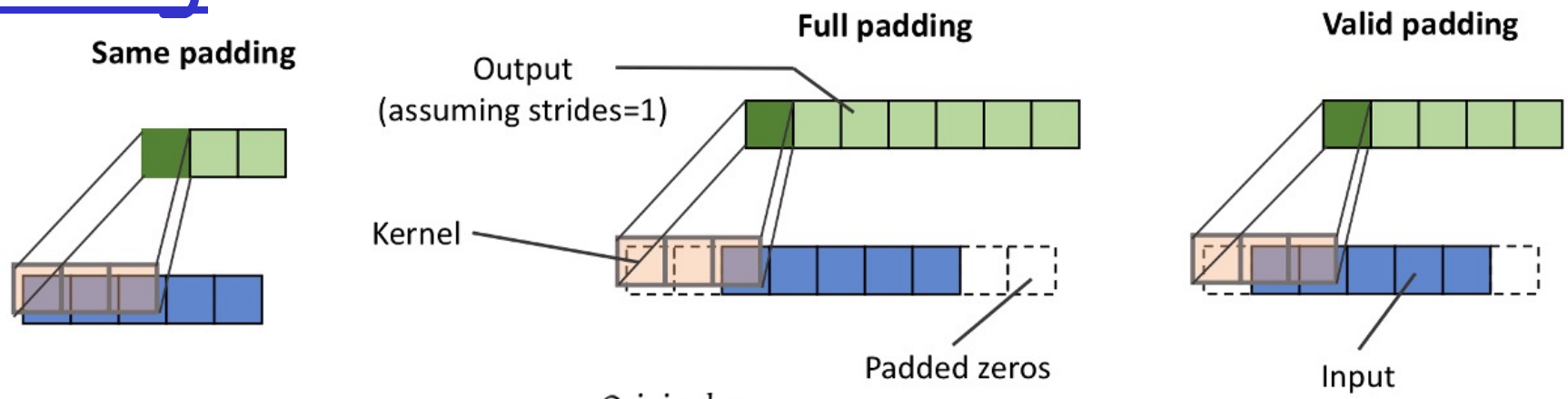
Features representing local patches

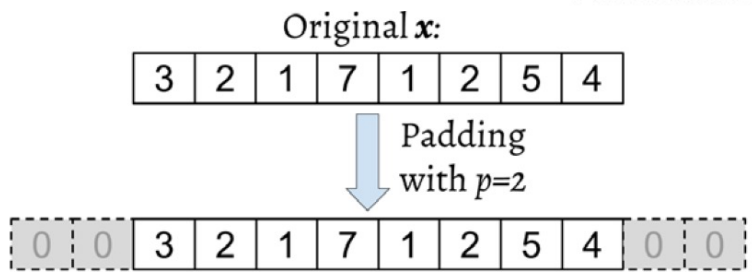
An Example

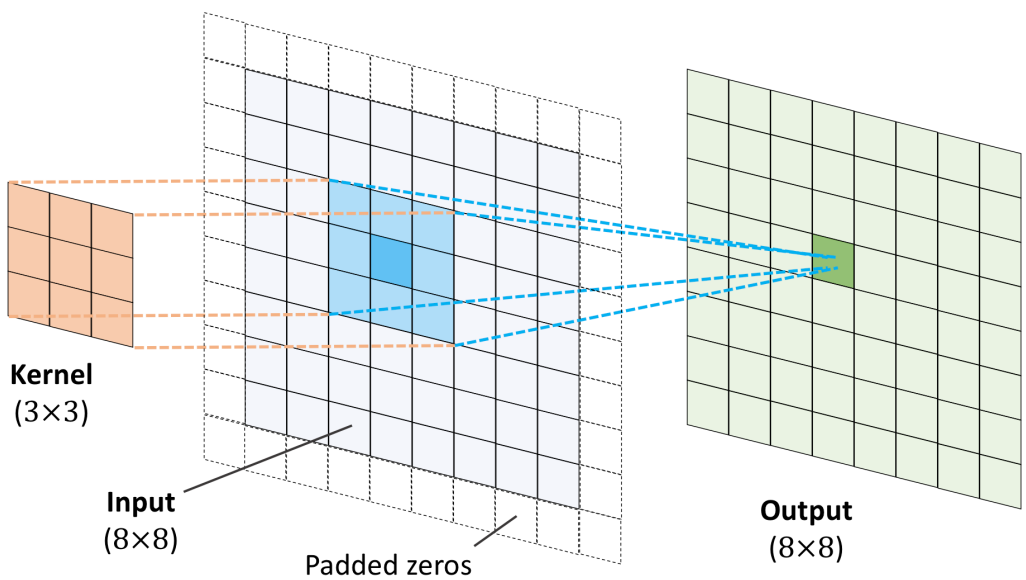
- Each pixel is subtracted by the value of its left neighboring pixel
  - Detect same features at multiple locations



Padding


Same padding
Full padding
Valid padding
Output
(assuming strides=1)
Kernel
Padded zeros
Input


Original x:
3 2 1 7 1 2 5 4
Padding
with p=2
0 0 3 2 1 7 1 2 5 4 0 0


Kernel
(3×3)
Input
(8×8)
Padded zeros
Output
(8×8)

Stride

x

3	2	1	7	1	2	5	4

$$y[0] = 3\times {}^{1}/_{4} + 2\times 1 + 1\times {}^{3}/_{4} + 7\times {}^{1}/_{2}$$

$$\rightarrow y[0] = 7$$

3	2	1	7	1	2	5	4
$^1/_4$	1	$^3/_4$	$^1/_2$				

$$y[1] = 1\times {}^{1}/_{4} + 7\times 1 + 1\times {}^{3}/_{4} + 2\times {}^{1}/_{2}$$

$$\rightarrow y[1] = 9$$

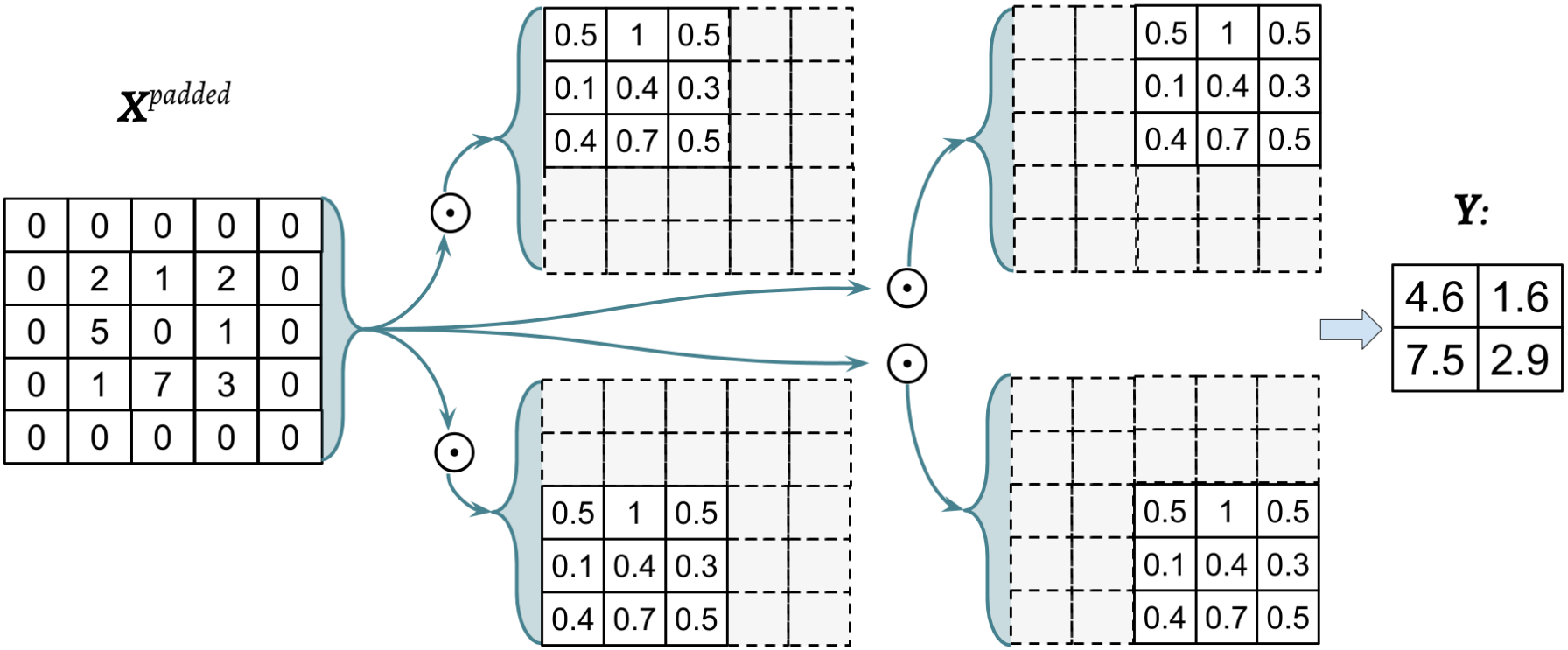
3	2	1	7	1	2	5	4
		$^1/_4$	1	$^3/_4$	$^1/_2$		

$$y[2] = 1\times {}^{1}/_{4} + 2\times 1 + 5\times {}^{3}/_{4} + 4\times {}^{1}/_{2}$$

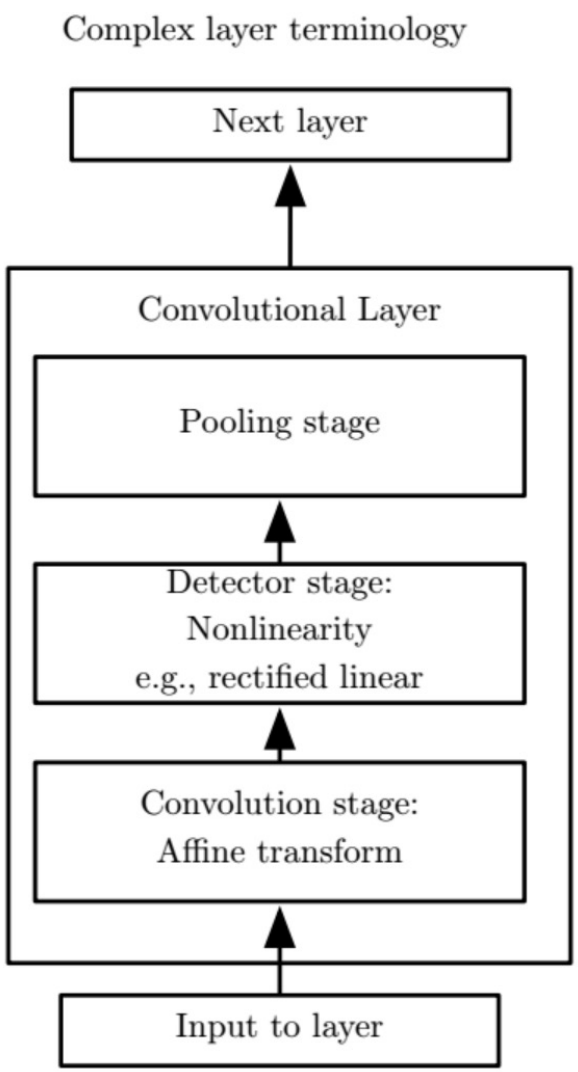
$$\rightarrow y[2] = 8$$

3	2	1	7	1	2	5	4
				$^1/_4$	1	$^3/_4$	$^1/_2$

Convolution

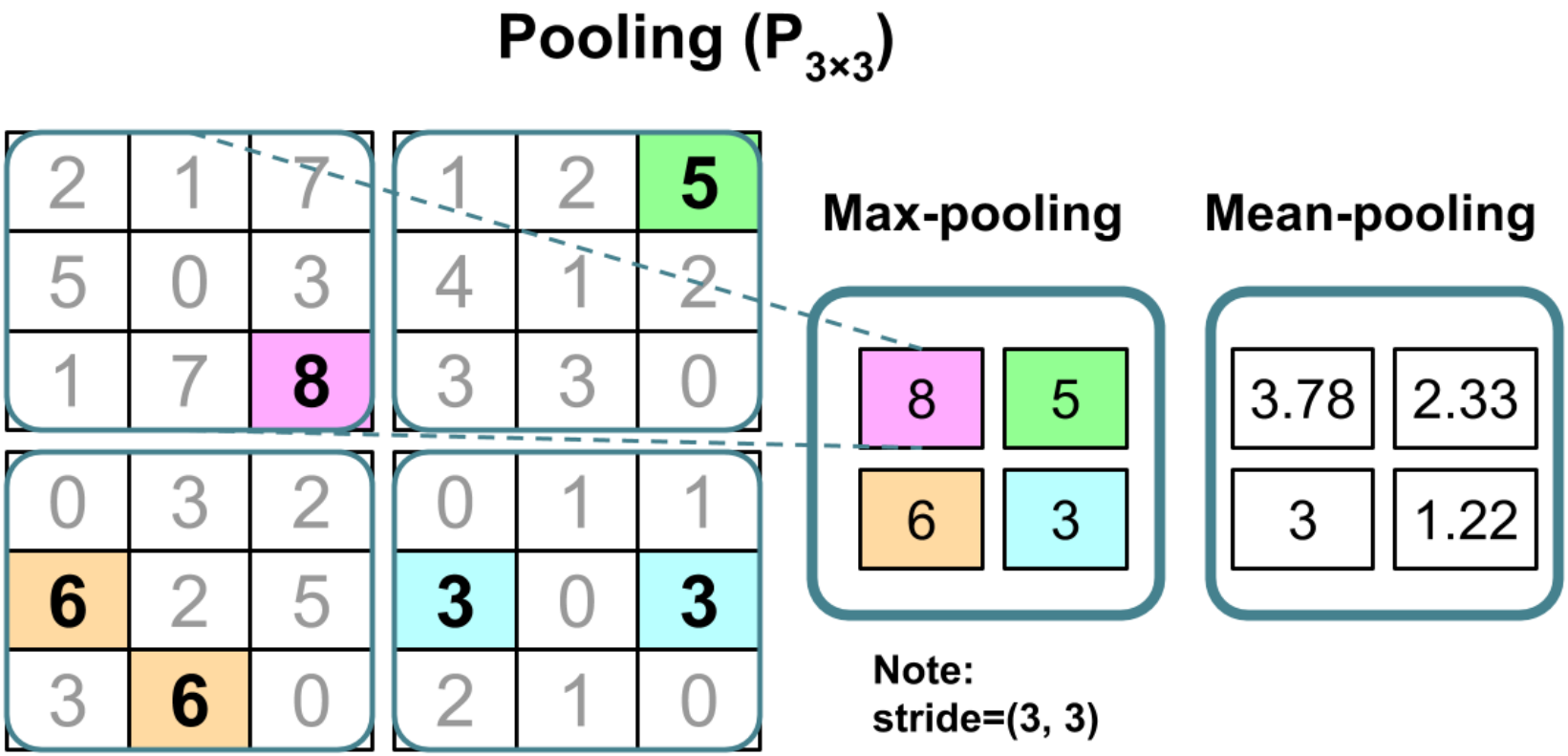


Pooling Stage



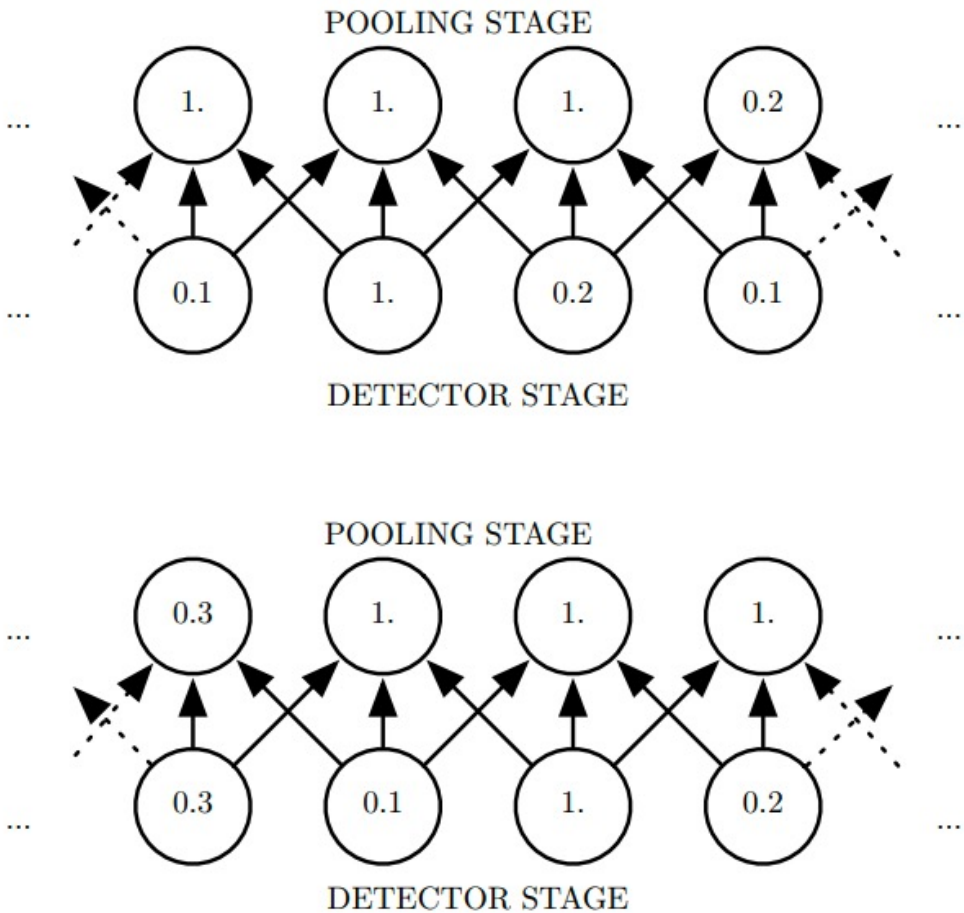
Pooling

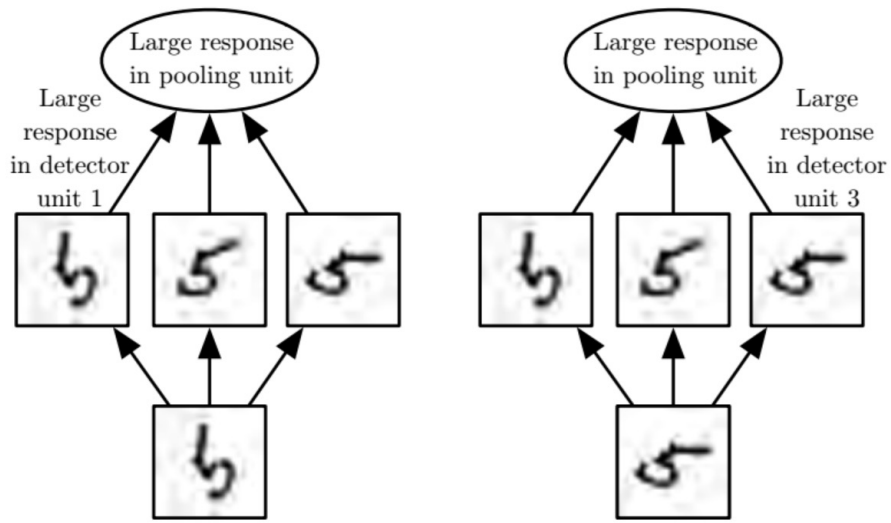
- A summary statistic of the nearby outputs



A way of reducing computation

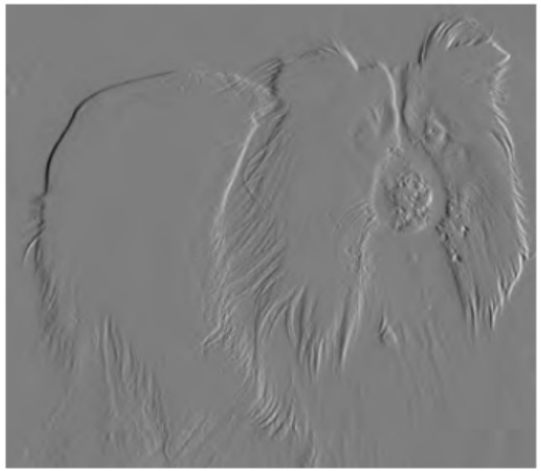
Pooling Invariance



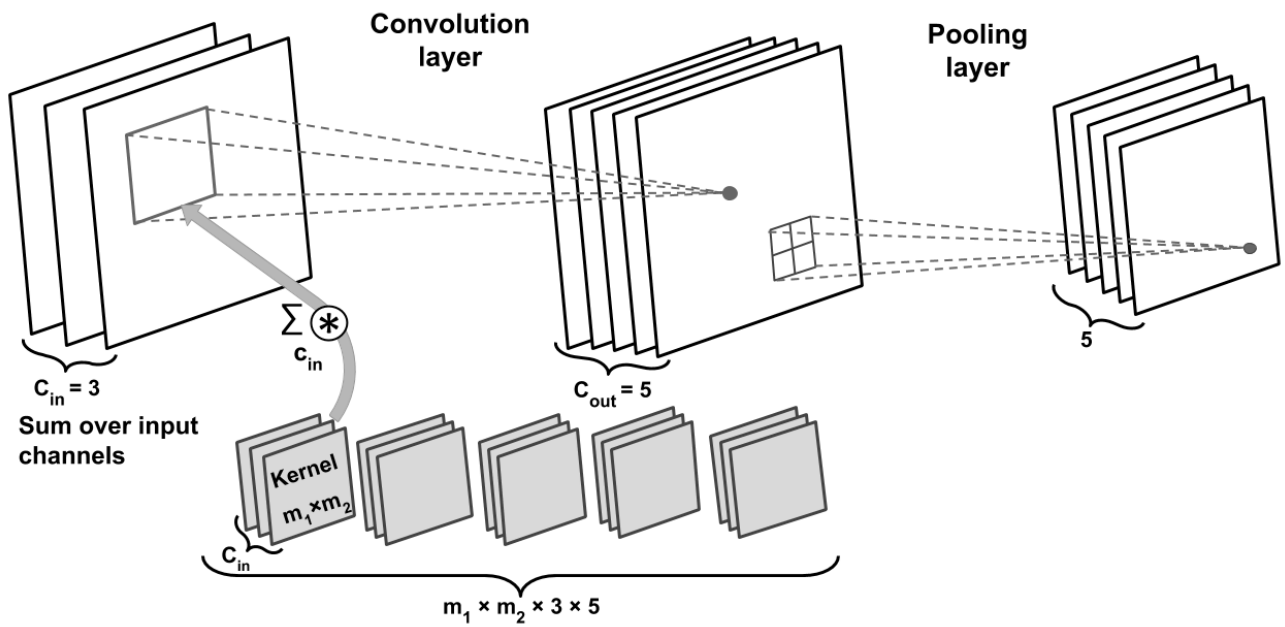


Channel

- Convolution and pooling
  - Zero probabilities on some parameters
  - Infinitely strong prior
  - Cause underfitting
  - Each kernel creates one feature map
  - One kind of features



Trainable Parameters



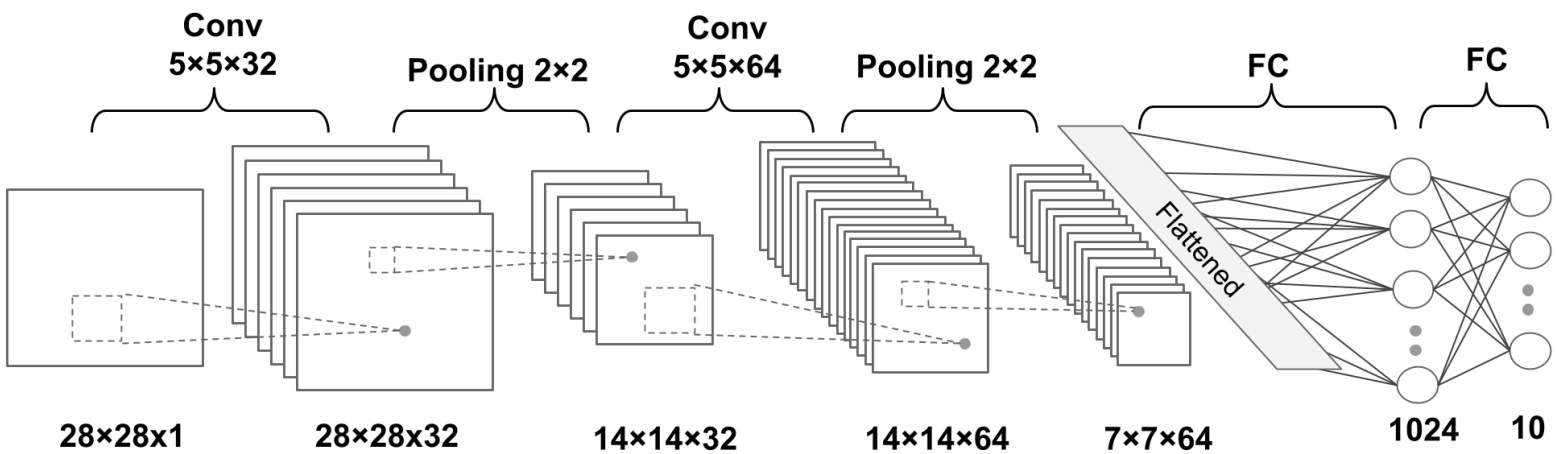
$$Z_{o,j,k} = \sum_{i,m,n} H_{i,j+m,k+n} K_{i,o,m,n} + b_o$$

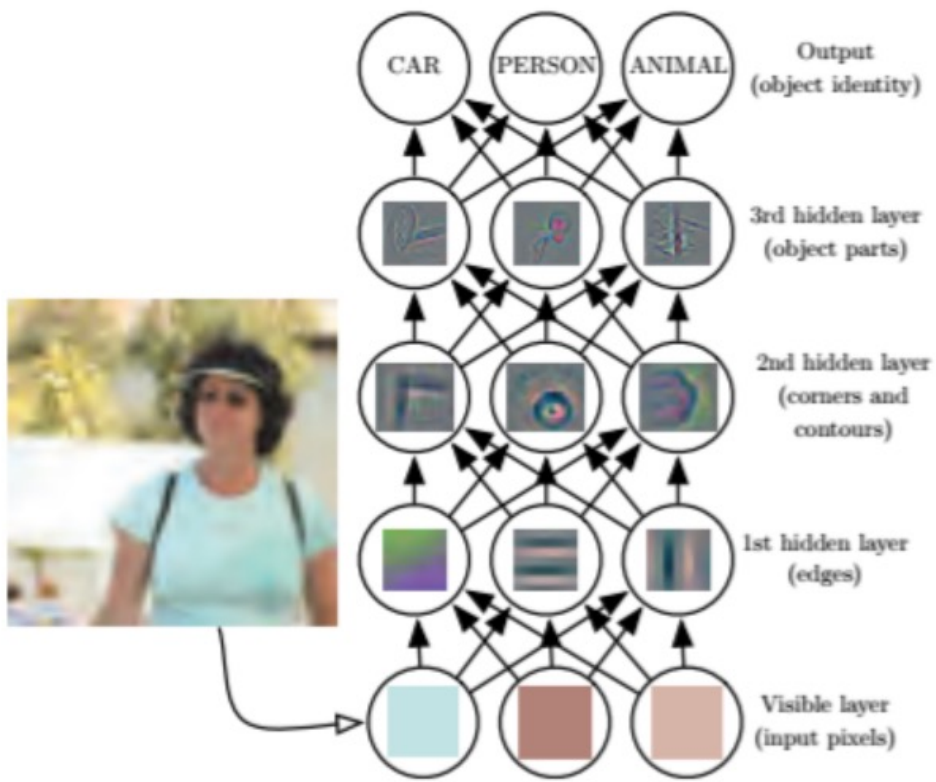
Output channel

Input channel

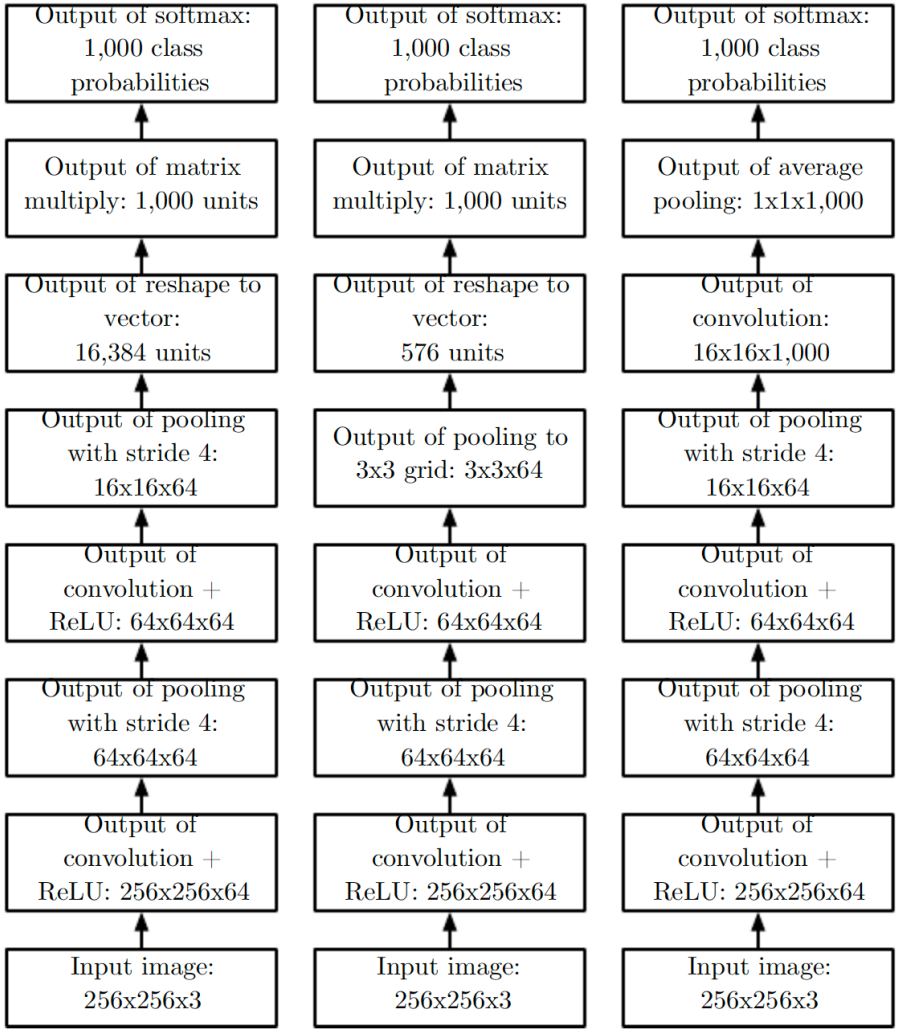
$$m_1 m_2 c_{in} c_{out} + c_{out}$$

Example

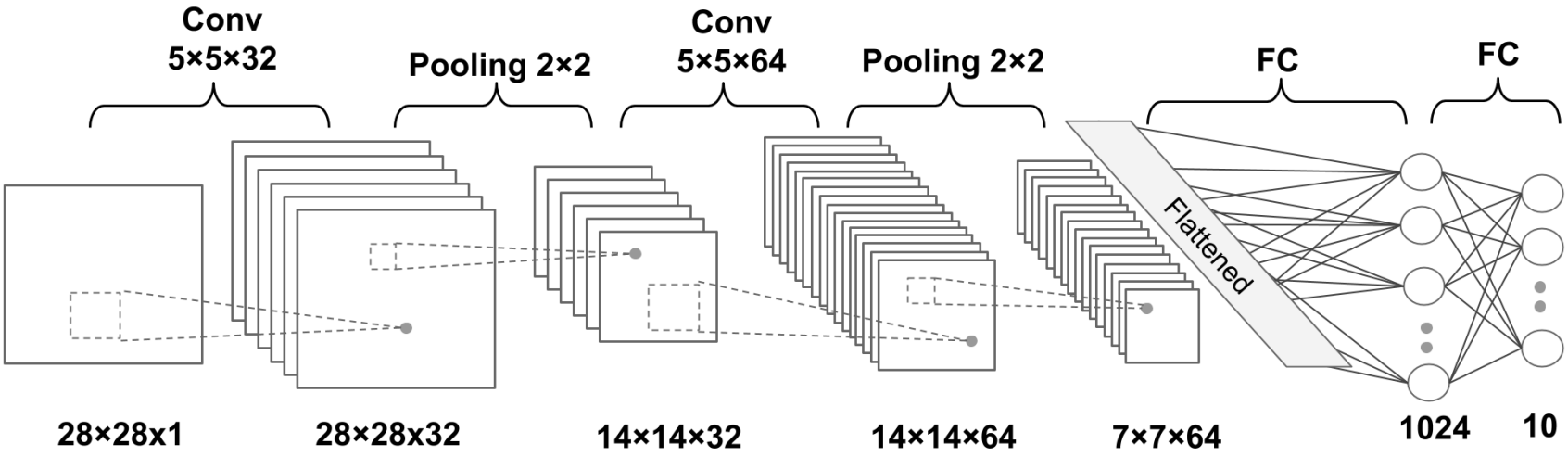

Conv
5×5×32
Pooling 2×2
Conv
5×5×64
Pooling 2×2
FC
FC
Flattened
28×28x1
28×28x32
14×14×32
14×14×64
7×7×64
1024
10


CAR
PERSON
ANIMAL
Output
(object identity)
3rd hidden layer
(object parts)
2nd hidden layer
(corners and
contours)
1st hidden layer
(edges)
Visible layer
(input pixels)

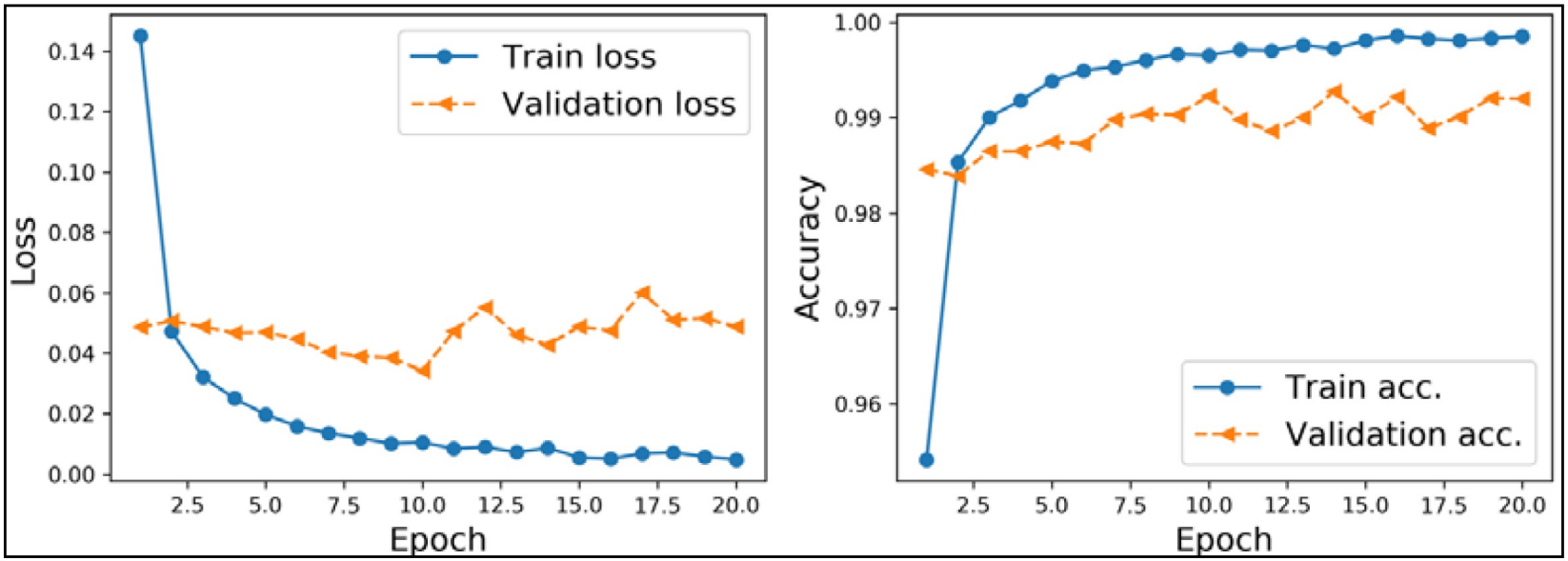
Architectures

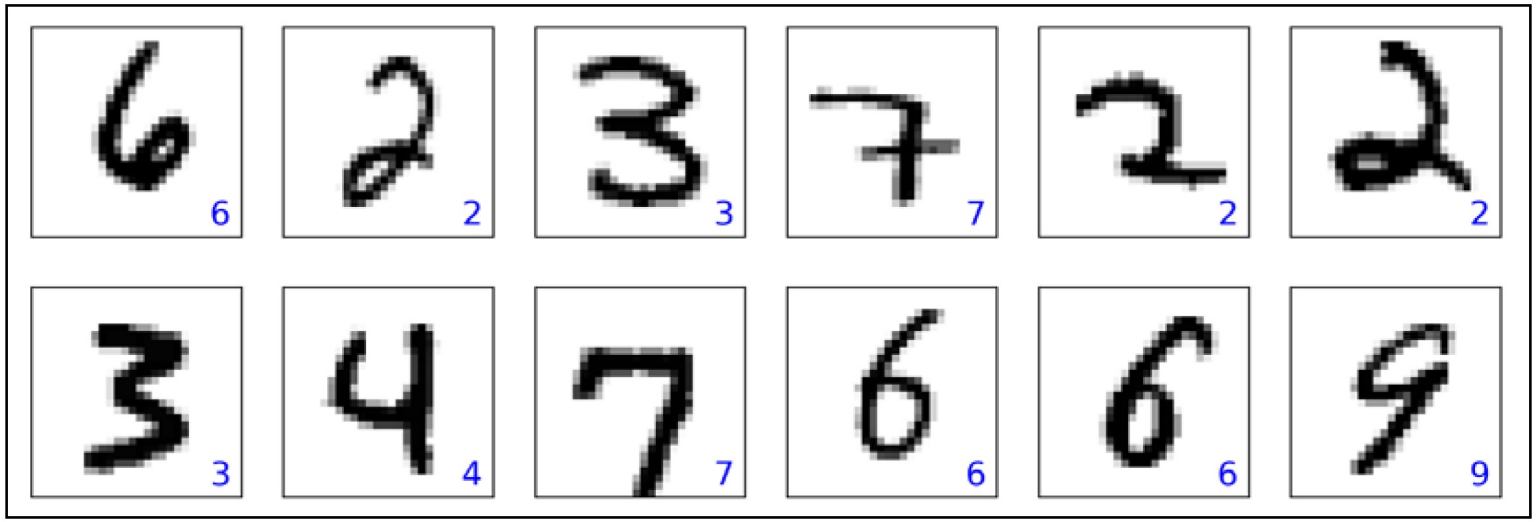


Example

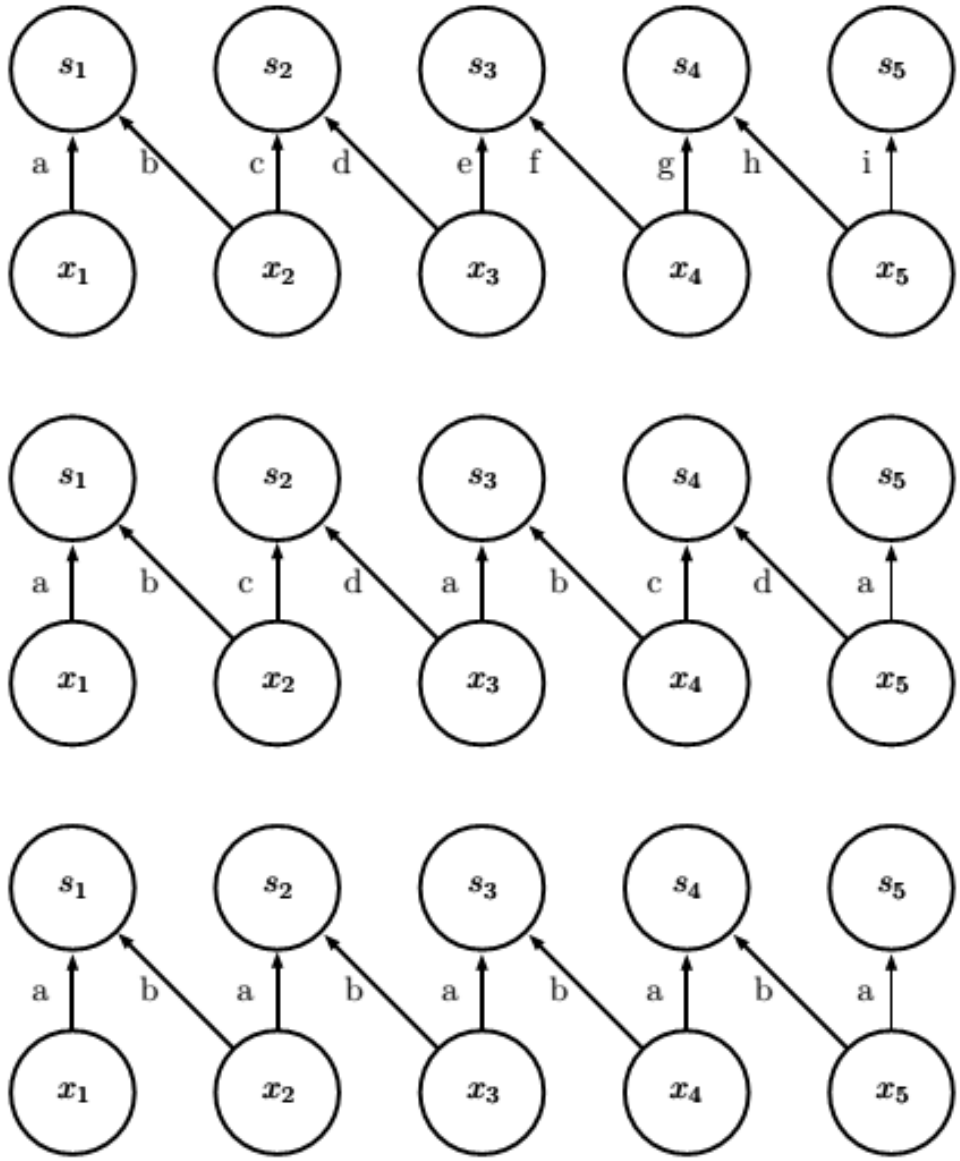

Conv
5×5×32
Pooling 2×2
Conv
5×5×64
Pooling 2×2
FC
FC
Flattened
28×28x1
28×28x32
14×14×32
14×14×64
7×7×64
1024
10

Results


Train loss
Validation loss
Loss
Epoch
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Train acc.
Validation acc.
Accuracy
1.00
0.99
0.98
0.97
0.96


6
2
3
7
2
2
3
4
7
6
6
9

CNN Variants



Outline

- Convolutional Neural Networks
- Recurrent Neural Networks
- Practical Methodology

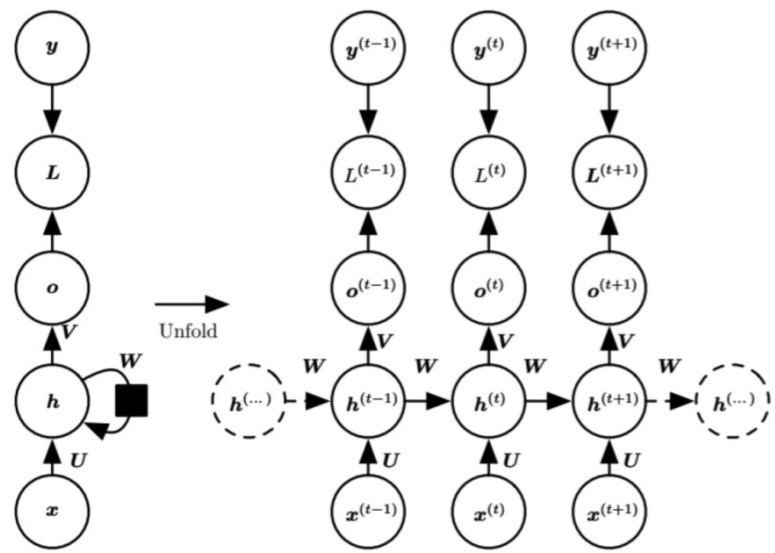
Introduction

- Sequence modeling
  - Features are indexed in time
  - Future depends on the past
- Assumption
  - Recursive hidden variables

$$\boldsymbol{h}^{(t)} = g\left(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \cdots, \boldsymbol{x}^{(1)}\right)$$
$$= f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \theta)$$

  - Parameter sharing

$$\boldsymbol{z}^{(t)} = \boldsymbol{b} + W\boldsymbol{h}^{(t-1)} + U\boldsymbol{x}^{(t)}$$
$$\boldsymbol{h}^{(t)} = \tanh(\boldsymbol{z}^{(t)})$$
$$\boldsymbol{o}^{(t)} = \boldsymbol{c} + V\boldsymbol{h}^{(t)}$$
$$\hat{y}^{(t)} = \mathrm{softmax}(\boldsymbol{o}^{(t)})$$



Training of RNN

- Assume a sequence of input-outputs

$$\left\{(x_i^{(t)}, y_i^{(t)}): 1 \le i \le D, 1 \le t \le \tau\right\} = \left\{(\boldsymbol{x}^{(t)}, \boldsymbol{y}^{(t)}): 1 \le t \le \tau\right\}$$

- Loss function

$$L = -\log p_\tau\left(\{\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, \cdots, \boldsymbol{y}^{(\tau)}\} | \{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(\tau)}\}\right)$$

$$= -\log \prod_{t=1}^{\tau} p\left(\boldsymbol{y}^{(t)} | \{\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, \cdots, \boldsymbol{y}^{(t-1)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(t)}\}\right)$$

$$= -\log \prod_{t=1}^{\tau} p\left(\boldsymbol{y}^{(t)} | \{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(t)}\}\right)$$

$$= -\sum_{t=1}^{\tau} \log p_{model}\left(y^{(t)} | \{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(t)}\}\right)$$

$$L^{(t)} = \log p\left(y^{(t)} | \{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(t)}\}\right)$$

$$L = -\sum_{t=1}^{\tau} \log p\left(y^{(t)} | \{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(t)}\}\right) = \sum_{t=1}^{\tau} L^{(t)}$$

Training of RNN

$$L = \sum_{t=1}^{\tau} L^{(t)} \qquad \frac{\partial L}{\partial L^{(t)}} = 1$$

$$\nabla_{\boldsymbol{o}^{(t)}} L = \frac{\partial L}{\partial L^{(t)}} \nabla_{\boldsymbol{o}^{(t)}} L^{(t)}$$

$$(\nabla_{\boldsymbol{o}^{(t)}} L)_i = \frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}} = \hat{y}_i^{(t)} - I_{i,y^{(t)}}$$

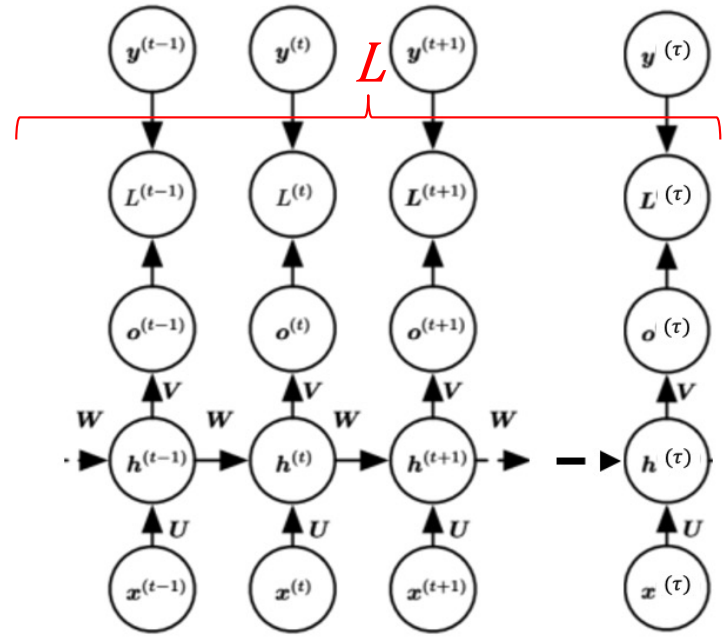
Recursive

$$\nabla_{\boldsymbol{h}^{(t)}} L = \left(\frac{\partial \boldsymbol{h}^{(t+1)}}{\partial \boldsymbol{h}^{(t)}}\right)^{\top} (\nabla_{\boldsymbol{h}^{(t+1)}} L) + \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}}\right)^{\top} (\nabla_{\boldsymbol{o}^{(t)}} L)$$

$$\nabla_{\boldsymbol{h}^{(\tau)}} L = V^{\top} \nabla_{\boldsymbol{o}^{(\tau)}} L$$

$$\frac{dh^{(t+1)}}{dz^{(t+1)}} = \frac{d\tanh(z^{(t+1)})}{dz^{(t+1)}} = (1 - tanh^2\,(z^{(t+1)})) = (1 - h^{(t+1)})$$

$$\nabla_{h^{(t)}} L = \boldsymbol{W}^{\top} (\nabla_{\boldsymbol{h}^{(t+1)}} L)\, \mathrm{diag}\left(1 - \left(\boldsymbol{h}^{(t+1)}\right)^2\right) + \boldsymbol{V}^{\top} (\nabla_{\boldsymbol{o}^{(t)}} L)$$



$$\begin{aligned}
\boldsymbol{z}^{(t)} &= \boldsymbol{b} + W\boldsymbol{h}^{(t-1)} + U\boldsymbol{x}^{(t)} \\
\boldsymbol{h}^{(t)} &= \tanh(\boldsymbol{z}^{(t)}) \\
\boldsymbol{o}^{(t)} &= \boldsymbol{c} + V\boldsymbol{h}^{(t)} \\
\hat{y}^{(t)} &= \mathrm{softmax}(\boldsymbol{o}^{(t)})
\end{aligned}$$

Training of RNN

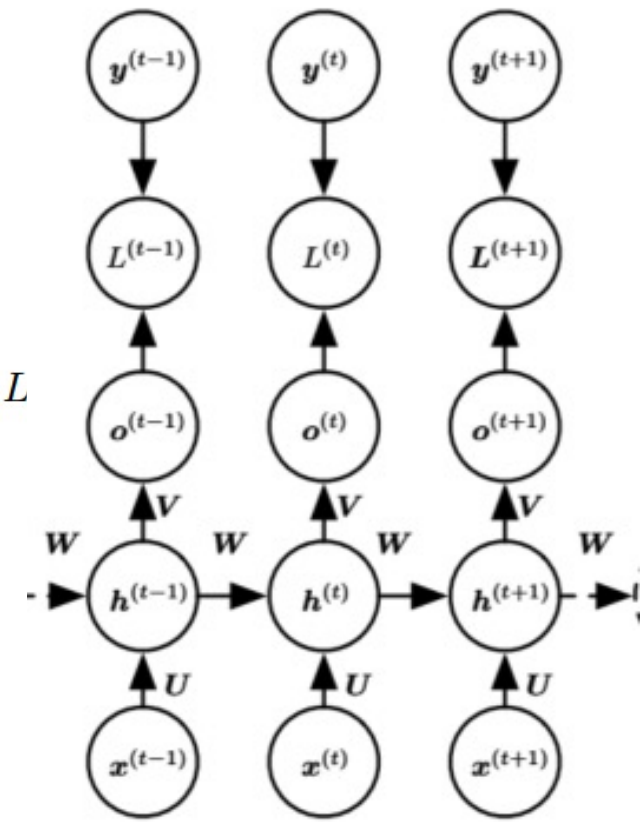
$$\nabla_{\boldsymbol{c}} L = \sum_t \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{c}}\right)^\top \nabla_{\boldsymbol{o}^{(t)}} L = \sum_t \nabla_{\boldsymbol{o}^{(t)}} L,$$

$$\nabla_{\boldsymbol{b}} L = \sum_t \left(\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{b}^{(t)}}\right)^\top \nabla_{\boldsymbol{h}^{(t)}} L = \sum_t \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t)}\right)^2\right) \nabla_{\boldsymbol{h}^{(t)}} L$$

$$\nabla_{\boldsymbol{V}} L = \sum_t \sum_i \left(\frac{\partial L}{\partial o_i^{(t)}}\right) \nabla_{\boldsymbol{V}} o_i^{(t)} = \sum_t \left(\nabla_{\boldsymbol{o}^{(t)}} L\right) \boldsymbol{h}^{(t)\top},$$

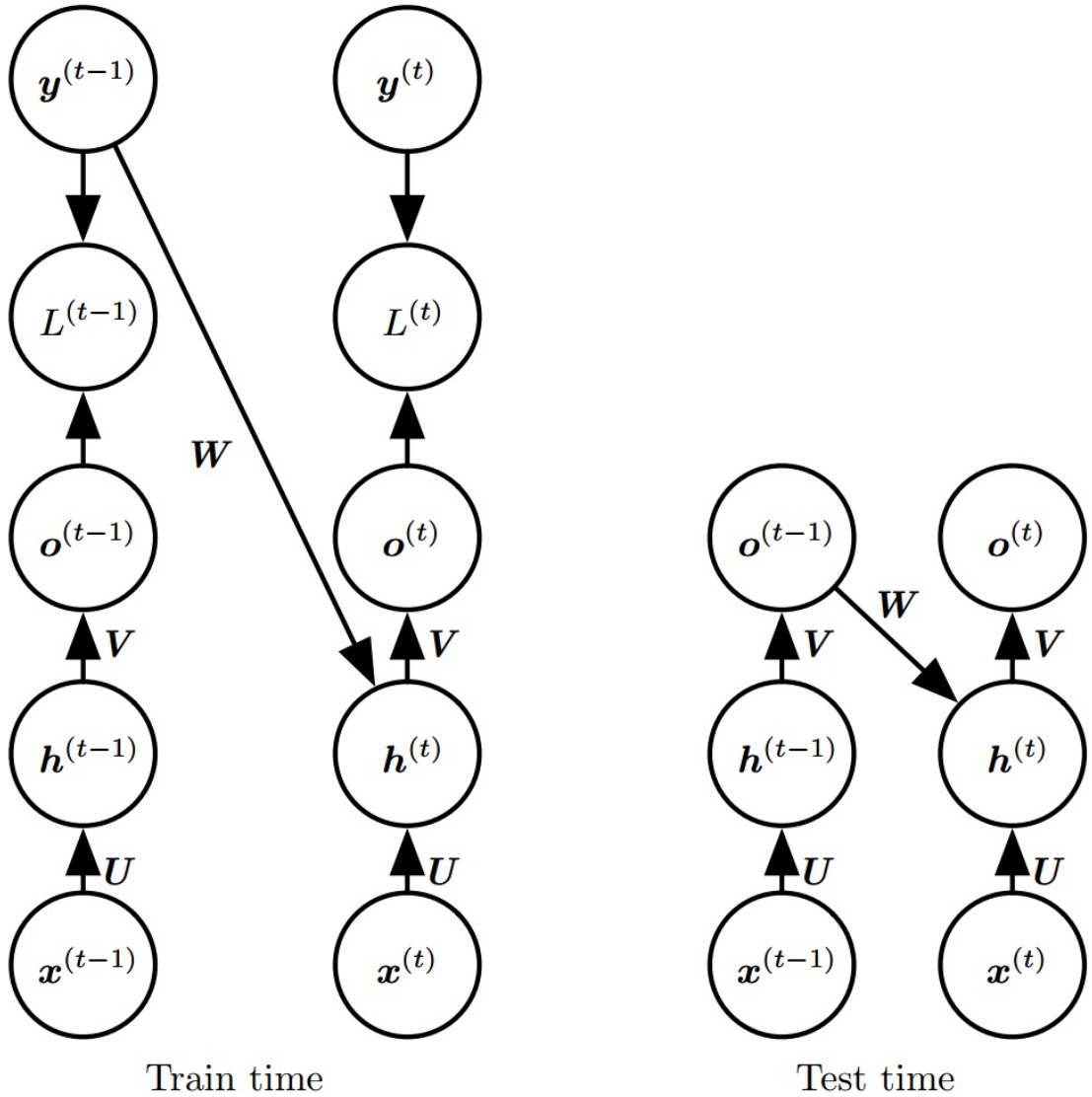
$$\begin{aligned}\nabla_{\boldsymbol{W}} L &= \sum_t \sum_i \left(\frac{\partial L}{\partial h_i^{(t)}}\right) \nabla_{\boldsymbol{W}^{(t)}} h_i^{(t)} \\ &= \sum_t \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t)}\right)^2\right) \left(\nabla_{\boldsymbol{h}^{(t)}} L\right) \boldsymbol{h}^{(t-1)\top},\end{aligned}$$

$$\begin{aligned}\nabla_{\boldsymbol{U}} L &= \sum_t \sum_i \left(\frac{\partial L}{\partial h_i^{(t)}}\right) \nabla_{\boldsymbol{U}^{(t)}} h_i^{(t)} \\ &= \sum_t \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t)}\right)^2\right) \left(\nabla_{\boldsymbol{h}^{(t)}} L\right) \boldsymbol{x}^{(t)\top},\end{aligned}$$

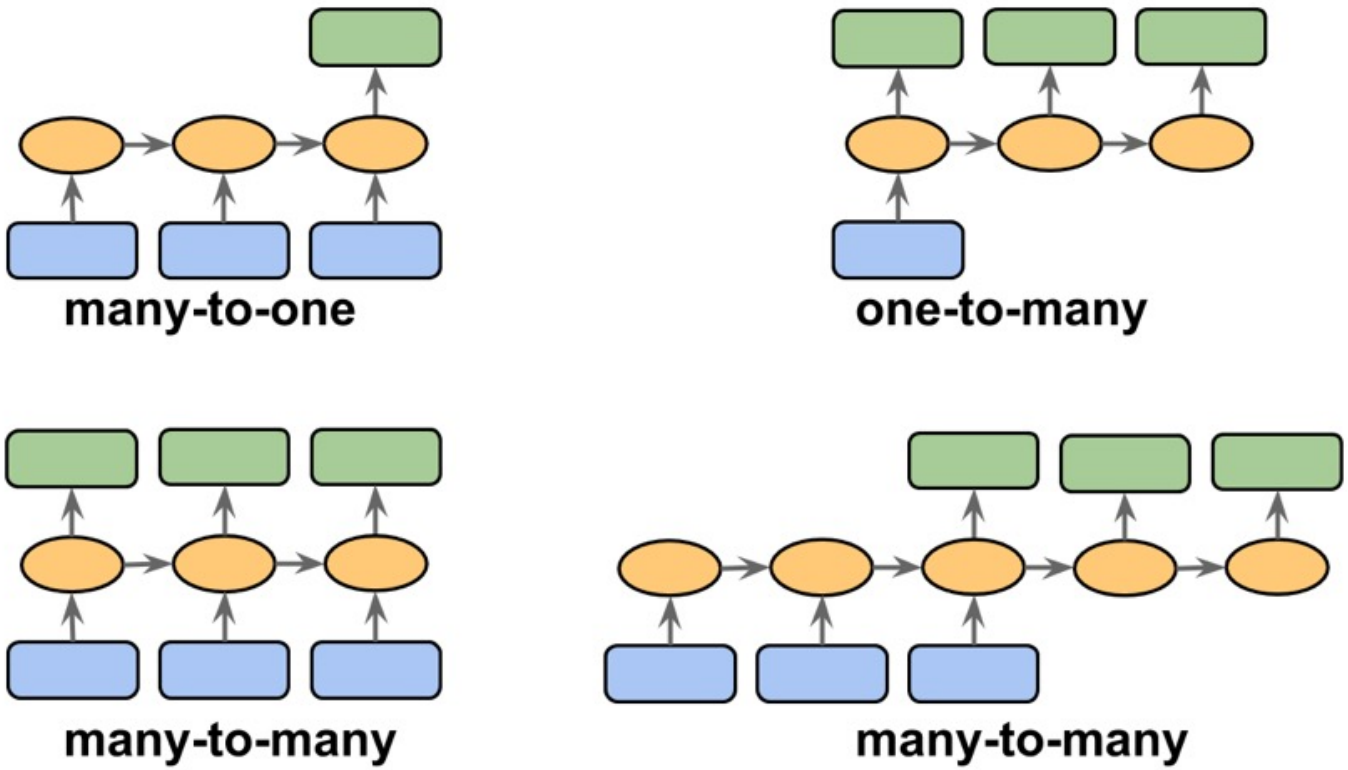


$$\boldsymbol{z}^{(t)} = \boldsymbol{b} + W\boldsymbol{h}^{(t-1)} + U\boldsymbol{x}^{(t)}$$
$$\boldsymbol{h}^{(t)} = \tanh(\boldsymbol{z}^{(t)})$$
$$\boldsymbol{o}^{(t)} = \boldsymbol{c} + V\boldsymbol{h}^{(t)}$$
$$\hat{y}^{(t)} = \operatorname{softmax}(\boldsymbol{o}^{(t)})$$

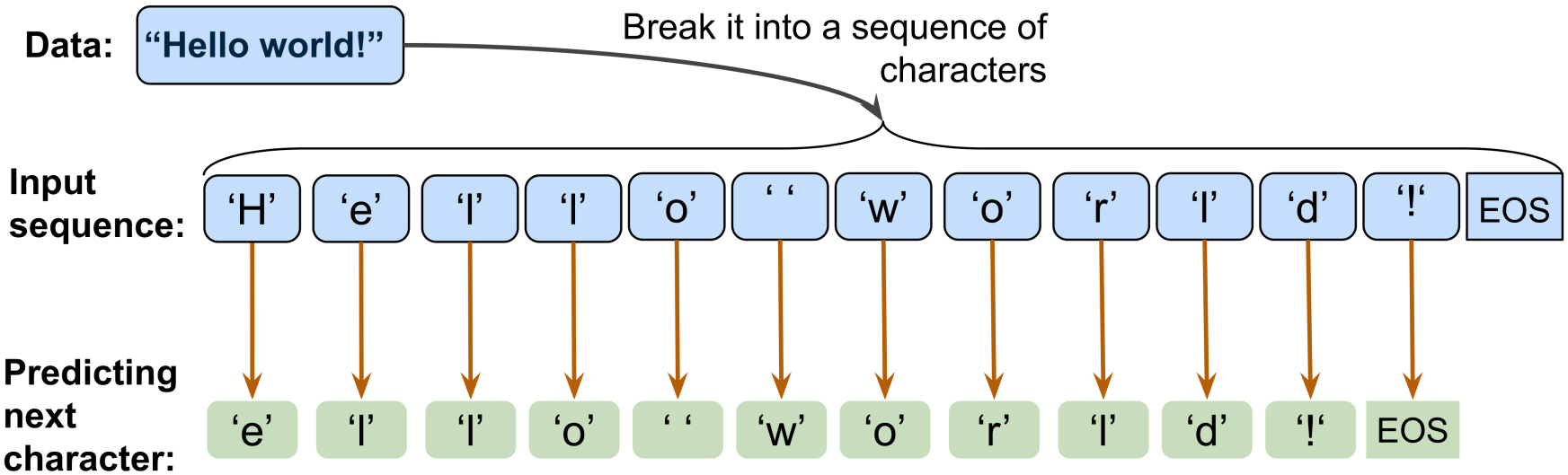
Teacher Forcing



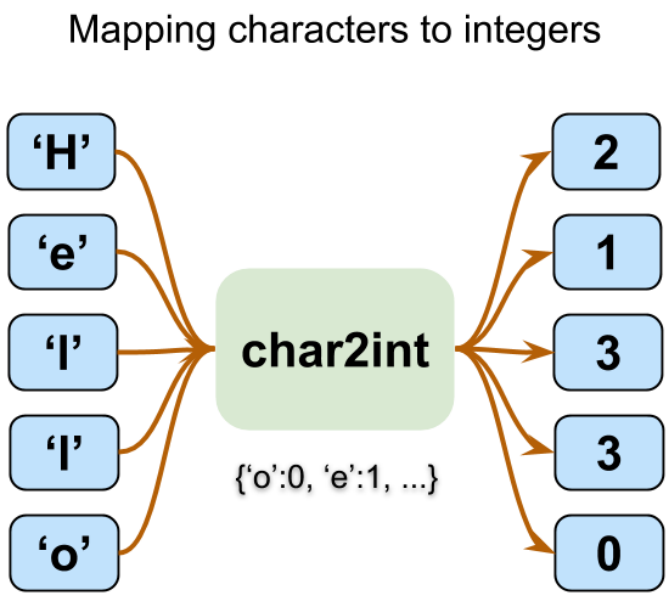
Applications

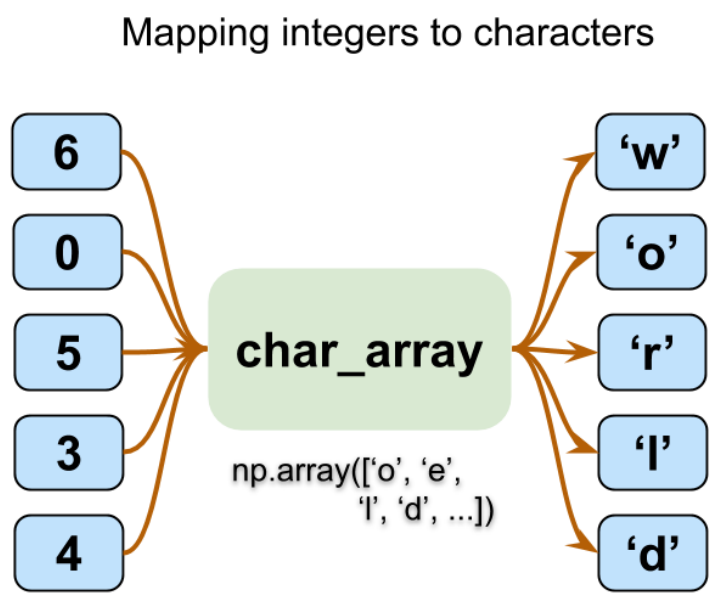


Example

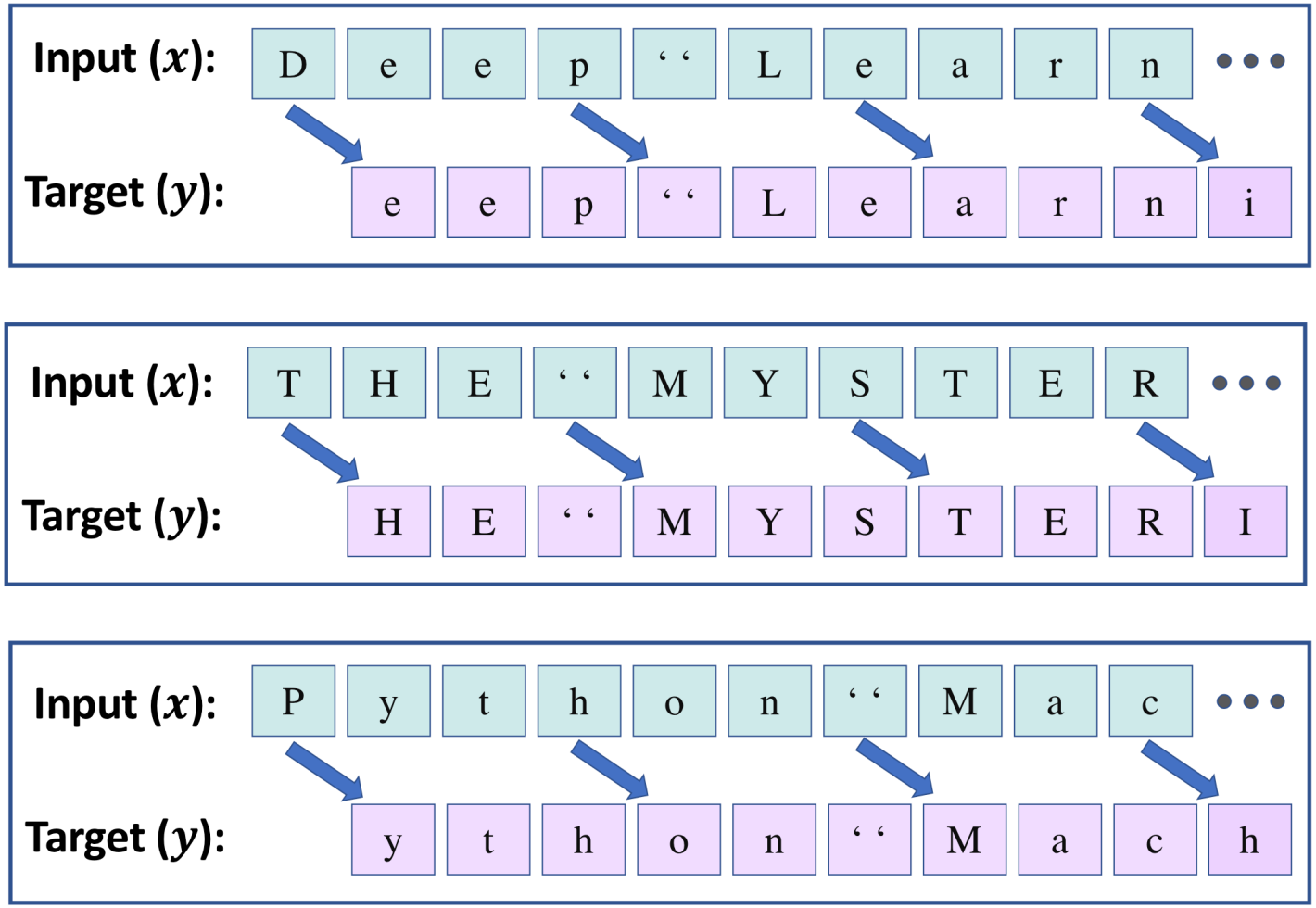

Data:
"Hello world!"
Break it into a sequence of characters
Input sequence:
'H' 'e' 'l' 'l' 'o' ' ' 'w' 'o' 'r' 'l' 'd' '!' EOS
Predicting next character:
'e' 'l' 'l' 'o' ' ' 'w' 'o' 'r' 'l' 'd' '!' EOS

Embedding


Mapping characters to integers
'H'
'e'
'l'
'l'
'o'
char2int
{'o':0, 'e':1, ...}
2
1
3
3
0


Mapping integers to characters
6
0
5
3
4
char_array
np.array(['o', 'e',
'l', 'd', ...])
'w'
'o'
'r'
'l'
'd'

Creating Training Examples



Challenges

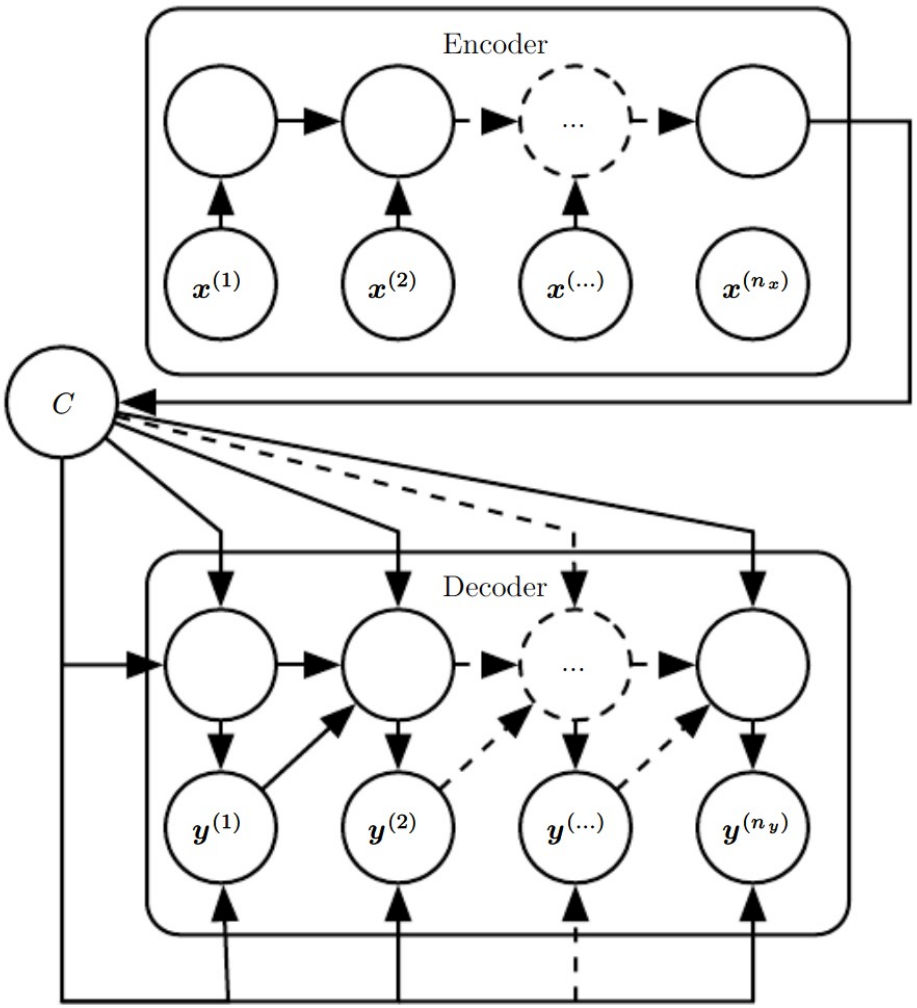
- Challenge
  - Long-term dependencies
    - Example
      - I grew up in France…I speak fluent French
      - Dave, as a result of his controversial claims, is now a pariah
    - Vanishing and exploding gradient due to many stages sharing the same weights

$$\boldsymbol{h}^{(t)} = (W^t)^\top \boldsymbol{h}^{(0)}$$

  - Inefficient sequential computation

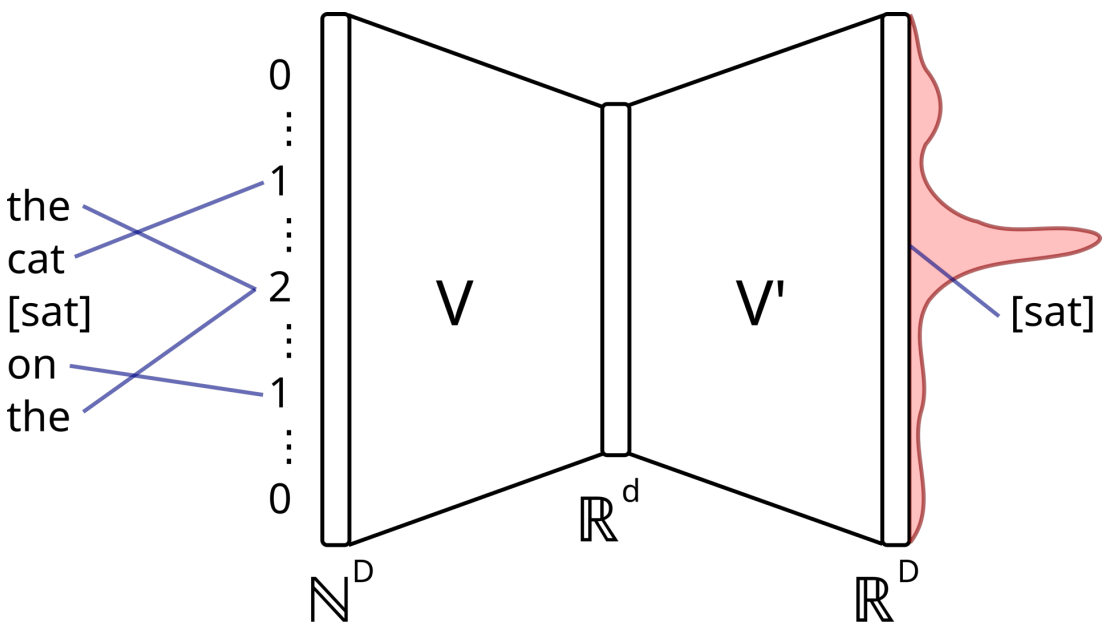
$$\nabla_{\boldsymbol{h}^{(t)}} L = \left(\frac{\partial \boldsymbol{h}^{(t+1)}}{\partial \boldsymbol{h}^{(t)}}\right)^\top (\nabla_{\boldsymbol{h}^{(t+1)}} L) + \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}}\right)^\top (\nabla_{\boldsymbol{o}^{(t)}} L)$$

Encoder-Decoder Architecture

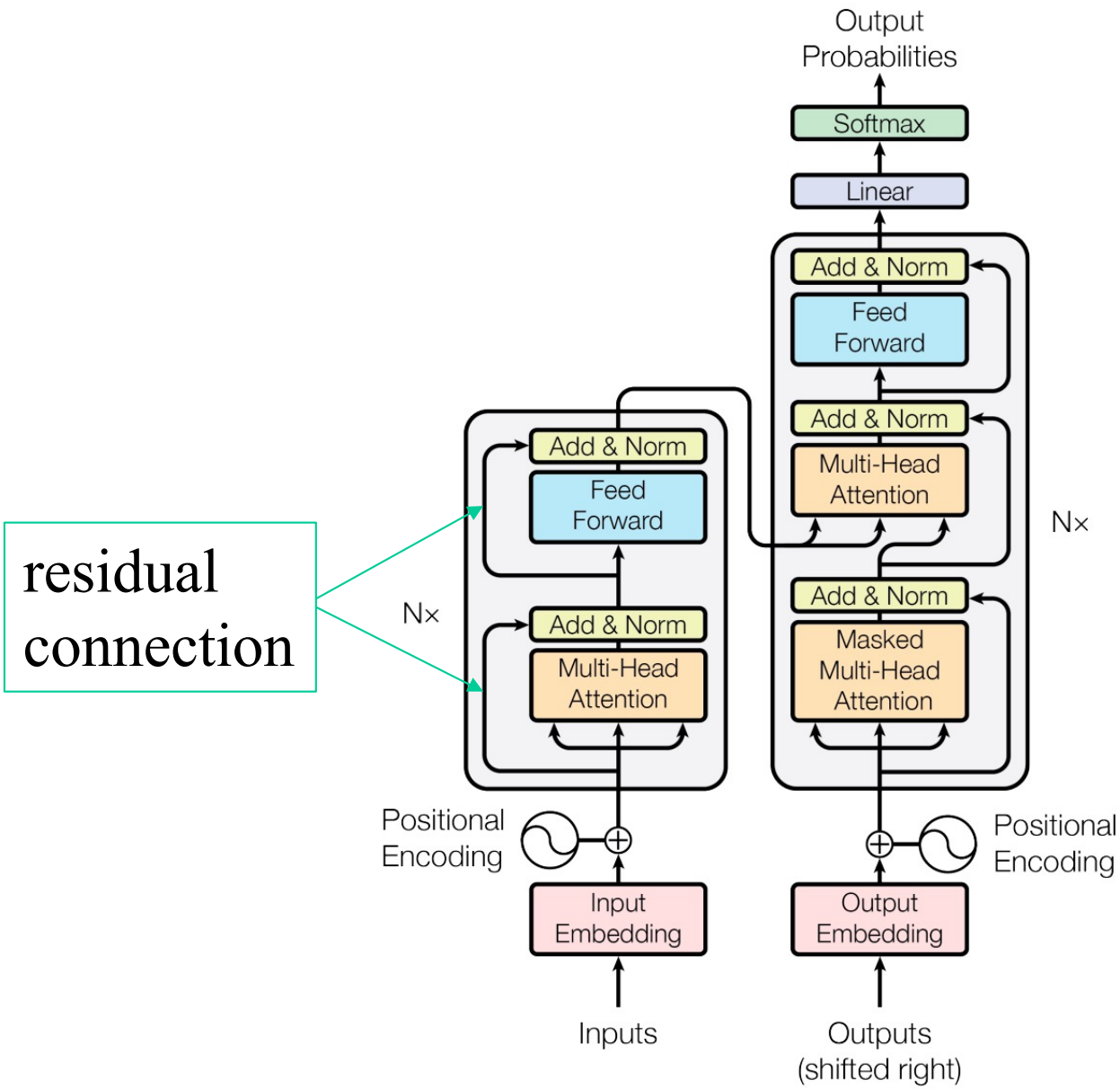


Embedding for LLM

- Embedding by considering neighborhood
- Identify each word by an integer w
- Map each word to a vector $w \to v \in \mathbb{R}^d$



Transformer Model



Self-Attention Mechanism

- Find correlation

$$\omega_{ij} = \boldsymbol{x}^{(i)^{\mathsf{T}}} \boldsymbol{x}^{(j)}$$

- Calculate the weight

$$W_{ij} = \frac{\exp(\omega_{ij})}{\sum_{j=0}^{T} \exp(\omega_{ij})} = softmax\left(\left[\omega_{ij}\right]_{j=0\ldots T}\right)$$

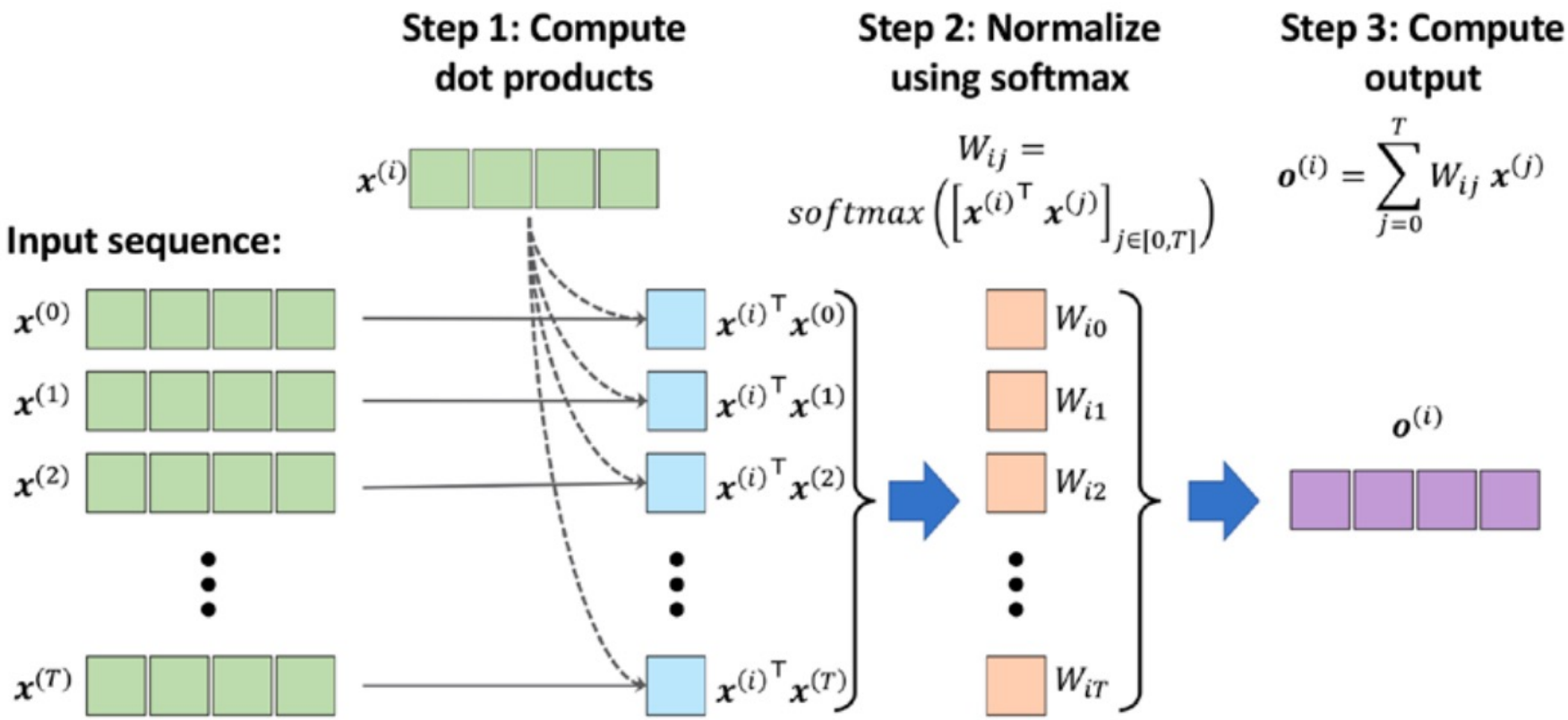
- Generate dependent output

$$\boldsymbol{o}^{(i)} = \sum_{j=0}^{T} W_{ij} \boldsymbol{x}^{(j)}$$

- Apply causal mask

$$\omega_{ij} = 0, j \geq i$$

Self-Attention Mechanism



Parameterizing Self-Attention

- Query sequence:

$$\boldsymbol{q}^{(i)} = U_q \boldsymbol{x}^{(i)}$$

- Key sequence:

$$\boldsymbol{k}^{(i)} = U_k \boldsymbol{x}^{(i)}$$

- Value sequence:

$$\boldsymbol{v}^{(i)} = U_v \boldsymbol{x}^{(i)}$$

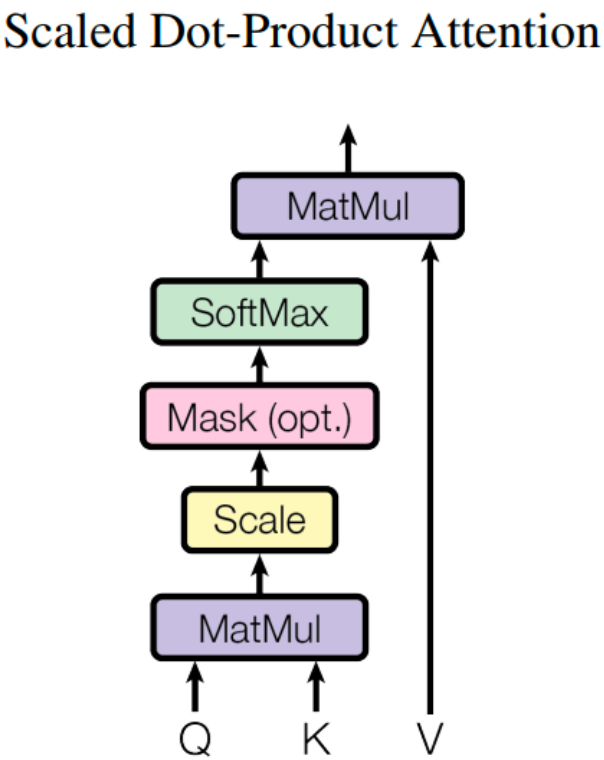
- Calculatina the weights:

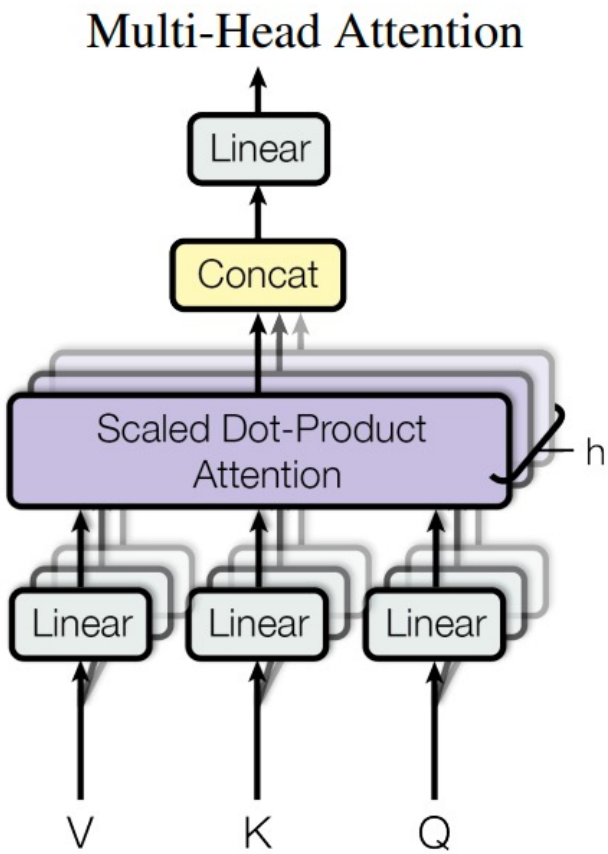
$$\omega_{ij} = \boldsymbol{q}^{(i)\top} \boldsymbol{k}^{(j)} \qquad m = d_k = d_v. \qquad W_{ij} = softmax\left(\frac{\omega_{ij}}{\sqrt{m}}\right)$$

- Generate the output:

$$\boldsymbol{o}^{(i)} = \sum_{j=0}^{T} W_{ij} \boldsymbol{v}^{(j)}$$

Multi-Head Attention





Positional Encoding

- To incorporate positional information into input data for parallel processing

$$f: X = \left(\boldsymbol{x}^{(1)}, \cdots, \boldsymbol{x}^{(M)}\right) \longrightarrow Z = (\boldsymbol{z}_1, \cdots, \boldsymbol{z}_M)$$

$$P = (\boldsymbol{p}_1, \cdots, \boldsymbol{p}_M)$$

$$E = (\boldsymbol{z}_1 + \boldsymbol{p}_1, \cdots, \boldsymbol{z}_M + \boldsymbol{p}_M)$$

$$p_{i,2j} = \sin\left(\frac{i}{10000^{\frac{2j}{d_{\text{model}}}}}\right)$$

$$p_{i,2j+1} = \cos\left(\frac{i}{10000^{\frac{2j}{d_{\text{model}}}}}\right)$$

Outline

- Convolutional Neural Networks
- Recurrent Neural Networks
- Practical Methodology

Parallel Processing

- CPU vs. GPU implementation
  - CPU: floating point vs. fixed point
    - Python vs. TensorFlow
  - GPU: multithreaded
    - Harder to program
- Large-scale distributed implementation
  - Data parallelism
    - Split dataset across devices
    - Replicate full model on each device
    - Synchronize gradients to update the shared model
  - Model parallelism
    - Each machine runs a different part of a model in pipelining

Other Techniques for Prediction

- Model compression
  - Use complex models to generate more data
  - Train a smaller model with new data added
- Dynamic structure
  - Also called conditional computation
  - Rare objects need more complex model
  - Use a cascade of models with increasing capacity
  - Can reject most requests earlier

Summary

- Key concepts

Parameter Sharing, Sparse Interconnections, Attention, Validation, Grid Search

- New models

CNN, RNN, LSTM, Transformer

References

- *Deep Learning*, I. Goodfellow, Y. Bengio, and A. Courville, MIT Press, 2016
- *Python Machine Learning,* S. Raschka and V. Mirjalili, Packt Publishing, 3rd Edition, 2019